Problem Statement:

The code demonstrates a basic e-healthcare system where patients can schedule appointments with specific types of doctors (eye or skin specialists). It prompts for patient information, determines the doctor based on the check type, displays doctor details, schedules an appointment, confirms it, and initiates the doctor's duties.

The system manages patients, doctors (eye, and skin specialists), appointments, and appointment confirmations. It handles patient details, doctor specialization, appointment scheduling, and confirmation.

Methodology and Solution:

Concepts (OOP) that were used in e-health management system:

Abstraction:

Abstract classes (MediPro) and interface (Confirm).

MediPro is an abstract class representing medical personnel with an abstract method performDuties(). The Confirm interface ensures the implementation of the confirmAppointment() method.

Inheritance:

Subclasses (Doctor, EyeDoctor, SkinDoctor) inheriting from a base class (MediPro).

Doctor is a subclass of MediPro, inheriting its properties and methods. EyeDoctor and SkinDoctor further specialize as subclasses of Doctor.

Encapsulation:

Private attributes in classes with public getter methods.

Attributes like name, dateTime, etc., are encapsulated within classes with private access, allowing controlled access using getter methods.

Polymorphism:

Method overriding (performDuties()).

performDuties() is an abstract method in MediPro overridden in the Doctor class, demonstrating polymorphic behavior in subclasses (EyeDoctor and SkinDoctor).

CODE:

Source Refactor Run Debug Profile Team Tools Window Help EHealthcareSystem - Apache NetBeans IDE 19

<default config> 198.3/342.0MB

...va SalEm.java ConstructorOverloading.java MethodOverloading.java Exceptiondemo.java Encapsulation.java Polymorphism.java Main.java EHealthcareSystem.java

Source History

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package ehealthcaresystem;
import java.util.Scanner;
// Abstract class for medical personnel
abstract class MediPro{
     protected String name;
 public MediPro(String name){
        this.name=name;
    }
 public abstract void performDuties(); // Abstract method to be implemented by subclasses
    // Getters and setters(Encapsulation)
 public String getName(){
     return name;
 }
}
```

```java
// Doctor class inheriting from MedicalPersonnel
class Doctor extends MediPro{
    private String sp;
    public Doctor(String name, String sp){
        super(name);
        this.sp=sp;
    }
public String getSpecialization(String sp){
   return sp;
}
   //override
    public void performDuties(){
        System.out.println(name + " will examine the patient.");
    }
    // Getter for specialization
    public String getSpecialization(){
        return sp;
    }
}
// Eye Doctor class inheriting from Doctor
class EyeDoctor extends Doctor{
    public EyeDoctor(String name, String sp){
        super(name, sp);
    }
}
// Skin Doctor class inheriting from Doctor
class SkinDoctor extends Doctor{
    public SkinDoctor(String name, String sp){
        super(name, sp);
    }
}
```

```java
}
// Patient class
class Patient{
    private String name;
    private int age;
    private String gender;
    private String checkType; // To specify the type of check
    public Patient( String name, int age, String gender, String checkType) {
        this.name=name;
        this.age=age;
        this.gender=gender;
        this.checkType=checkType;
    }
    // Getters
    public String getName(){
        return name;
    }
    public int getAge(){
        return age;
    }
    public String getGender(){
        return gender;
    }
    public String getCheckType(){
        return checkType;
    }
}
```

```java
// Appointment class
class Appointment{
    private String dateTime;
    private Patient patient;
    private MediPro doctor;
    private String reason;

public Appointment( String dateTime, Patient patient, MediPro doctor, String reason){
        this.dateTime=dateTime;
        this.patient=patient;
        this.doctor=doctor;
        this.reason=reason;
    }
    // Getters
    public String getDateTime(){
        return dateTime;
    }
    public Patient getPatient(){
        return patient;
    }
    public MediPro getDoctor(){
        return doctor;
    }
    public String getReason(){
        return reason;
    }
}
```

```java
// Interface for confirming appointments
interface Confirm{
    void confirmAppointment();
}
// AppointmentConfirmation class implementing Confirmable interface
class AppointmentConfirmation implements Confirm{
   //override
    public void confirmAppointment(){
        System.out.println(x: "Appointment confirmed.");
    }
}
public class EHealthcareSystem{
    public static void main(String[] args){
        Scanner scanner=new Scanner(source:System.in);
```

```java
        // Input for patient
        System.out.println(x: "Enter patient details:");
        System.out.print(s: "Enter  name of the patient: ");
        String patientName=scanner.nextLine();
        System.out.print(s: "Enter  age of the patient: ");
        int patientAge=scanner.nextInt();
        scanner.nextLine(); // Consume newline
        System.out.print(s: "Enter  gender of the patient: ");
        String patientGender=scanner.nextLine();
        System.out.print(s: "Enter check type (eye/skin): ");
        String checkType=scanner.nextLine();
        // Creating patient1 object
        Patient p1 = new Patient(name: patientName,age:patientAge,gender:patientGender,checkType);
        // Creating different doctors
        EyeDoctor eyeDoctor=new EyeDoctor(name: "Dr.Rayhan kabir",sp: "Eye doctor");
        SkinDoctor skinDoctor=new SkinDoctor(name: "Dr.Mahmuda Hasan",sp: "Skin Doctor");
        // Displaying doctor information and confirming appointment based on patient's choice
        if(p1.getCheckType().equalsIgnoreCase(anotherString:"eye")) {
            displayDoctorInfo(d: eyeDoctor);
            confirmAppointment(p1, d1: eyeDoctor);
        }else if(p1.getCheckType().equalsIgnoreCase(anotherString:"skin")){
            displayDoctorInfo(d: skinDoctor);
            confirmAppointment(p1, d1: skinDoctor);
        }else{
            System.out.println(x: "No appointment scheduled because you can only take appointmrnt of eye and skin doctor.");
        }
    }
```

```java
    // Display doctor's information
    public static void displayDoctorInfo(Doctor d){
        System.out.println("\n Details of doctor:");
        System.out.println("Doctor: " + d.getName() + " (" + d.getSpecialization() + ")");
    }
    // Confirm the appointment based on patient's choice
    public static void confirmAppointment(Patient p1, Doctor d1){
        Appointment a1 = new Appointment("2024-1-5 10:00 PM",p1,d1,"Regular checkup");
        // Displaying appointment information
        System.out.println("\nAppointment Details:");
        System.out.println("Patient: " +a1.getPatient().getName());
        System.out.println("Doctor: " +a1.getDoctor().getName());
        System.out.println("Date & Time: " +a1.getDateTime());
        System.out.println("Reason: " +a1.getReason());
        // Confirming the appointment using abstraction
        Confirm confirmable=new AppointmentConfirmation();
        confirmable.confirmAppointment();
        // Performing doctor's duties
        a1.getDoctor().performDuties();
    }
}
```

ehealthcaresystem.EHealthcareSystem > main > if (p1.getCheckType().equalsIgnoreCase("eye"))

Algorithm Design:

1. **Class MediPro (Abstract Class):** Represents medical personnel.

Contains:

name (protected): Name of the medical professional.

MediPro(String name): Constructor to set the name.

abstract void performDuties(): Abstract method for performing duties.

2. **Class Doctor (Extends MediPro):** represents a doctor

Contains:

sp (private): Specialization of the doctor.

Doctor(String name, String sp): Constructor to set name and specialization.

String getSpecialization(): Getter for specialization.

void performDuties(): Overrides performDuties() from MediPro.

3.Class EyeDoctor and SkinDoctor (Subclasses of Doctor):

EyeDoctor and SkinDoctor are specific types of doctors inheriting from the Doctor class..

4.Class Patient: Represents patient information.

Contains:

Attributes for name, age, gender, and checkType.

Getters for accessing the attributes.

5.Class Appointment: Manages appointment details.

Contains:

Attributes for dateTime, patient, doctor, and reason.

Getters for accessing the attributes.

6.Confirm (Interface): Contains an abstract method confirmAppointment().

7.Class AppointmentConfirmation (Implements Confirm): Implements confirmAppointment() method.

8. Main Class (EHealthcareSystem):

main() Method:

1. Takes user input for patient details (**name**, **age**, **gender**, **checkType**).
2. Creates a **Patient** object (p1) based on the input.
3. Creates instances of **EyeDoctor** and **SkinDoctor**.
4. Determines the doctor based on the patient's **checkType.**

   -If the patient's check type is "eye," display and confirm the appointment with the eye doctor.
   - If the check type is "skin," display and confirm the appointment with the skin doctor.
   - Otherwise, inform the user that appointments are only available for eye and skin doctors.
5. Displays doctor information and confirms the appointment according to the **checkType**.
6. Displays appointment details including patient, doctor, date/time, reason.
7. Confirms the appointment and invokes doctor duties.

This algorithm provides a structure for managing patient appointments with specialized doctors in an e-healthcare system.

Execution Flow:

User Input**:** Gathers patient details using Scanner.

Doctor Selection**:** Determines the suitable doctor (EyeDoctor or SkinDoctor) based on the patient's checkType.

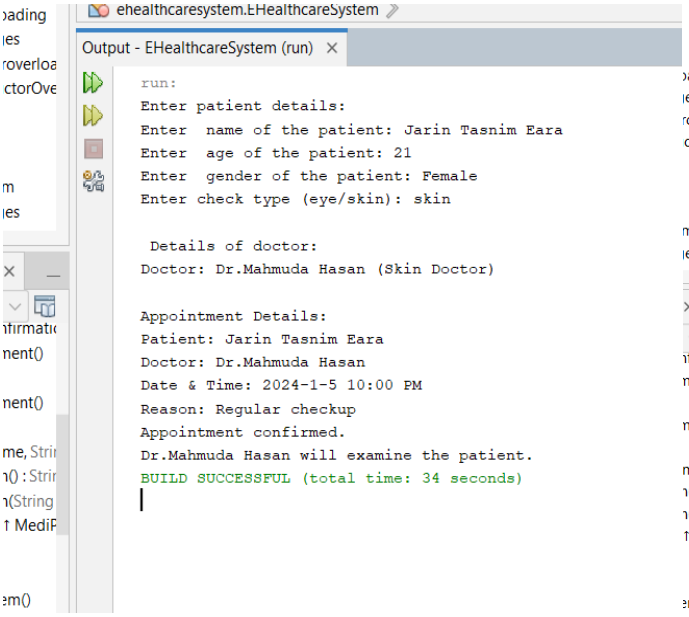
Appointment Handling**:** Creates an Appointment object with patient and doctor details.

Appointment Confirmation**:** Confirms the appointment using an instance of AppointmentConfirmation.

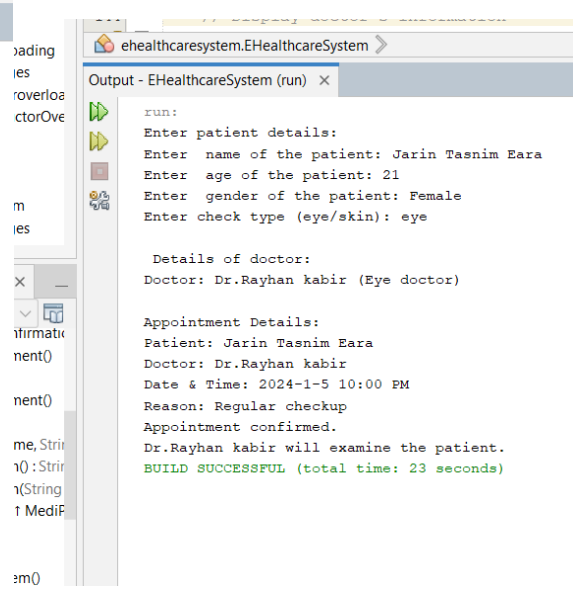
Doctor's Duties**:** Calls the performDuties**()** method of the appointed doctor.

Output Snap:

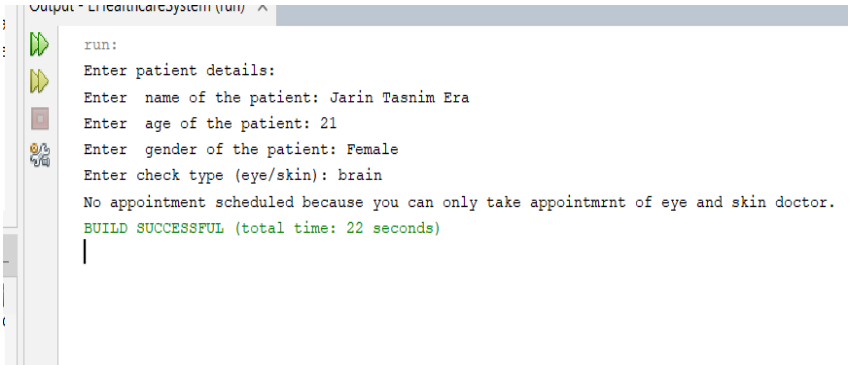
here are 3 types of output according to the given condition.



Snap-1: (checktype skin)



Snap-2: (checktype eye)



Snap-3 : (Checktype none)

Conclusion:

The code establishes a basic framework for an eHealthcare system, managing patients, doctors, appointments, and confirmation processes. It employs OOP principles to create a structured system, encapsulating data and functionalities within classes and leveraging inheritance and interfaces for modularity and extensibility.

Modern Tools Use:

This Java code appears to be a simple healthcare system implemented using object-oriented programming concepts. It includes classes for medical professionals (abstract and concrete), patients, appointments, and appointment confirmation. The code takes patient details as input, allows the selection of a specific doctor based on the type of check (eye or skin), and confirms the appointment.to solve the problem I used NetBeans(Tool).

To make this code work in NetBeans:

1. Open NetBeans.

2. Create a new Java project.

3. Create a new Java class file for each class in the provided code.

4. write the code into the corresponding class files within the NetBeans project.

5. Ensure that the main method (in the EHealthcareSystem class) is correctly set as the application entry point.

6. Run the project.

NetBeans allows us to execute our code directly within the IDE, by giving an interface to interact with and test the program. NetBeans provides immediate feedback on syntax errors, potential bugs, or warnings, aiding in code refinement. The IDE integrates various tools, making it easier to manage projects, navigate code, and ensure proper class interactions.

By using NetBeans, I can efficiently manage, develop, and test Java applications. It offers an environment that enhances coding productivity, facilitates debugging, and helps in drawing conclusions about the code's functionality and performance. That is why I used NetBeans to Create e-health management system.